Práctico 03 — Primitivas paramétricas y la matriz de modelo

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

4 de septiembre de 2026



Roadmap de la clase

Antecedentes

- Unidad V — malla poligonal
- Unidad VI — transformaciones
- El cubo: el vértice como *tupla de atributos*

Hoy

- Qué cambia cuando cambia la tupla
- Cómo se **genera** una malla matemáticamente
- La **matriz de modelo** y el `uniform`

Conectado a

- Unidad VII — proyección (teóricos 09 y 16-sep)
- Unidad VIII — caras ocultas (07-oct)
- Unidad IX — iluminación (21-oct)

Las cajas negras de hoy

- Va a quedar una **constante** en el vertex shader, más chica que la del práctico anterior: es debida a la proyección.
- Las **coordenadas de textura** se calculan y entran en el vértice pero no se utilizan aún. Servirá cuando veamos texturas.
- La **normal** se calcula y entra en el vértice pero no se utiliza aún. Servirá cuando veamos iluminación.

Quedó pendiente una pregunta de la clase anterior

Un **cilindro de 8 gajos**, con un color por gajo.

¿Cuántos vértices hacen falta para la superficie lateral?

La única regla que hace falta:

Se comparte un vértice **si y sólo si** coinciden **todos** sus atributos.

La nueva tupla

La propuesta para el cubo

```
struct Vertex
{
    float px, py, pz;  // posicion
    float r,  g,  b;   // color
};
```

La nueva propuesta

```
struct Vertex
{
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 tex_coords;
};
```

- Se elimina el **color**: no es un atributo del vértice, es del **objeto**.
- Se agrega la **normal**: la dirección perpendicular a la superficie en el vértice. Es un dato que se utiliza para calcular el sombreado (Unidad IX).
- Se agregan las **coordenadas de textura**: permiten mapear los colores a aplicar al objeto desde una imagen en la memoria.

Comparación entre tuplas

¿Cuántos vértices se necesitan para el cilindro con la *nueva* tupla?

	tupla vieja posición + color	**tupla nueva** posición + normal + uv
Cubo	24 el color cambia en la arista	**24** la normal cambia en la arista
Cilindro, 8 gajos (superficie lateral)	32 el color cambia en cada gajo	**18** la normal varía de forma continua

- El **cubo mantiene 24 vértices**: dos caras que se tocan en una arista tienen normales distintas y debido a eso no pueden unificarse los vértices.
- El **cilindro baja casi a la mitad**: la normal varía de forma continua, así que dos gajos vecinos **sí** pueden compartir el borde y por lo tanto unificar los vértices.

El contenido (datos) de la tupla afecta cómo definir la cantidad mínima de vértices.

¿Dónde va el color?

El color del vértice no forma parte de la tupla.

¿Cómo se asigna ahora?

El shader puede recibir datos con otro mecanismo que no sea por medio de un VBO.

Datos que no cambian por vértice

Motivación: Una pieza de color constante tiene **el mismo color para todos sus vértices**. Entonces hay que repetir el mismo valor muchas veces (cientos o miles).

uniform

Una variable del programa de shaders que **se fija antes de ejecutar el pipeline** y queda **constante durante toda la ejecución del mismo**. Mantiene el mismo valor para todos los vértices y todos los fragmentos de esa llamada.

```
// fragment shader
uniform vec3 uColor;      // <- el color de la pieza, uno por objeto

void main()
{
    FragColor = vec4(uColor, 1.0);
}
```

Atributo: un valor *por vértice*. **Uniform**: un valor *por ejecución del pipeline*.

Ampliar Shader

Un método por cada *tipo* de `uniform`

```
class Shader
{
public:
    // ... compile_from_source(), use(), id()

    void set_uniform(const std::string& nombre, const glm::mat4& m) const;
    void set_uniform(const std::string& nombre, const glm::vec3& v) const;
    void set_uniform(const std::string& nombre, float valor) const;
};
```

```
void Shader::set_uniform(const std::string& nombre, const glm::mat4& m) const
{
    const int loc = glGetUniformLocation(id_, nombre.c_str());
    glProgramUniformMatrix4fv(id_, loc, 1, GL_FALSE, &m[0][0]);     // DSA
}
```

No es una lista cerrada: cada tipo de `uniform` que haga falta, agrega una sobrecarga.

Ampliar Shader

Segunda versión de los setters para `uniforms`

La siguiente llamada **busca por texto** dentro del programa linkeado.

```
const int loc = glGetUniformLocation(id_, nombre.c_str());
```

Se repite una vez por cada `uniform` que haya y **por cada cuadro**: con 60 cuadros por segundo y varias piezas, son miles de búsquedas de `string` por segundo para obtener **siempre el mismo número**.

```
// se consulta el identificador una vez al crear el shader y se almacena
int loc(const std::string& nombre) const;

// y despues se usa el identificador y no el texto
void set_uniform(int ubicacion, const glm::mat4& m) const;
```

La versión por nombre da **legibilidad** al código;
la versión por ubicación **reduce la carga del bucle de dibujado**.

Cada modelo tiene su propio sistema de referencia

- Las coordenadas de los vértices de un modelo vienen dadas respecto al sistema de referencia que se haya definido al crearlo (arbitrario).
- Ej: `primitives::cube(...)` devuelve un cubo **centrado en el origen y alineado con los ejes X,Y,Z**.
- Para componer una escena se necesita *transformar* los modelos para adecuar el tamaño y ubicarlos en la escena (orientación y posición).

Matriz de modelo

Transformación que lleva un objeto de **su** sistema de coordenadas —donde fue generado— al sistema de coordenadas de **la escena**.

Permite modificar proporciones, orientación y posición del modelo.

Malla generada una vez $\longrightarrow$ muchas piezas distintas

cada una con su propia matriz de modelo

Construcción usando `glm`

`glm` (*OpenGL Mathematics*): biblioteca de C++ **sólo de encabezados** con los tipos y las operaciones de álgebra lineal de GLSL —`vec2/3/4`, `mat3/4`—. No es parte de OpenGL: es una dependencia más del proyecto.

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>

glm::mat4 model = glm::mat4(1.0f);             // identidad

model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0,0,1));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
```

`glm` no modifica: devuelve

`glm::translate(model, v);` **no hace nada** si no se asigna el resultado. Lo mismo para otras funciones de `glm`: `normalize`, `rotate` y `scale`.

`glm::mat4(1.0f)` construye la identidad. `glm::mat4()` **no** garantiza identidad: conviene escribir siempre el `1.0f`.

El orden de las transformaciones no es conmutativo

Rotar y después trasladar

```
model = T * R;

// con glm:
m = glm::translate(m, t);
m = glm::rotate(m, ang, eje);
```

La pieza gira **sobre sí misma** y queda en su lugar.

Trasladar y después rotar

```
model = R * T;

// con glm:
m = glm::rotate(m, ang, eje);
m = glm::translate(m, t);
```

La pieza **orbita** alrededor del origen.

Ninguno de los dos está mal. Producen resultados diferentes.

Cómo se escribe en `glm`

- En `model * v` se aplica primero la matriz que está **más a la derecha**.
- Cada llamada a `glm` **multiplica por derecha**: la última llamada que se escribe es la primera que se aplica.
- Para *escalar* → *rotar* → *trasladar* se escriben `translate`, `rotate` y `scale` **en ese orden**.

Matriz modelo en el shader

```
#version 460 core

layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoords;

uniform mat4 uModel;       // <- la matriz de este modelo
uniform mat4 uAjuste;      // <- ajuste de vista y proyección (caja negra por ahora)

void main()
{
    gl_Position = uAjuste * uModel * vec4(aPos, 1.0);
}
```

- El formato de vértice ahora declara **tres** atributos, no dos.
- `uModel` es el mismo para los (cientos, miles) de vértices de la pieza.
- `uAjuste` sigue siendo una caja negra

Datos fijos vs generación automática

Los datos de la malla del cubo del Práctico 02 son 24 vértices **escritos a mano**.

¿Qué pasaría si quisiésemos escribir los datos para un cilindro?

	vértices	**datos a escribir**
Cubo	24	24
Cilindro de 8 gajos	36	36
Cilindro de 36 gajos	148	148

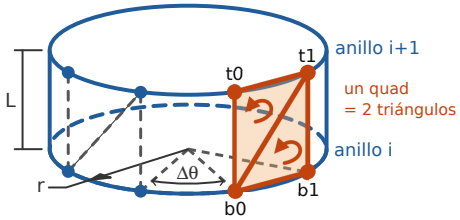
En el caso del cilindro, el problema es la discretización polar (gajos) que requiere recalcular todos los vértices cuando se modifica.

Las mallas de algunas figuras geométricas conviene **generarlas** a partir de unos pocos parámetros que la caracterizan.

Malla poligonal de un cilindro

Datos del cilindro:

- Radio (ó diámetro)
- Largo
- Cantidad de gajos (discretización diametral)
- Cantidad de anillos (discretización axial - opcional)

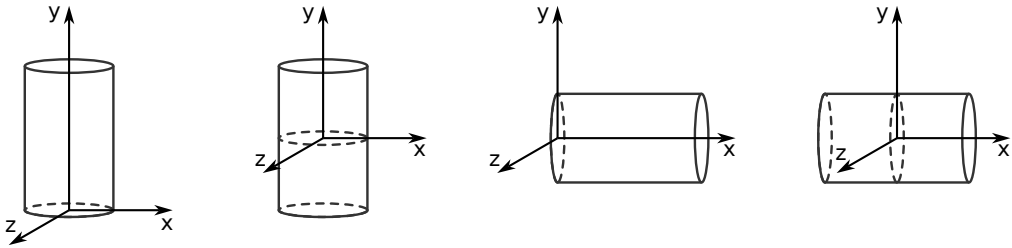


Pasos:

1. Definir la ubicación del sistema de referencia del modelo (importante!)
2. Discretizar los **anillos** para generar los vértices (gajos)
3. Usando los *vértices* en los anillos generar una tira de cuadriláteros definiendo triángulos

Malla poligonal de un cilindro

Paso 1 — Definir el sistema de referencia



Todas estas definiciones son correctas, pero hay que elegir una al plantear la parametrización.

Malla poligonal de un cilindro

Paso 2 — Discretizar los anillos

```
for (unsigned i = 0; i <= N; ++i)           // OJO: <= y no <
{
    const float th = 2.0f * PI * float(i) / float(N);

    Vertex v;
    v.position   = { r*cos(th), y, r*sin(th) };
    v.normal     = {   cos(th), 0,   sin(th) };
    v.tex_coords = { float(i)/float(N), v_del_anillo };
    vertices.push_back(v);
}
```

Por qué `<= N` y no `< N`

En la costura —donde el último gajo se encuentra con el primero— la posición y la normal **coinciden**, pero la coordenada de textura vale **0 y 1 a la vez**. Un atributo difiere, así que **el vértice se duplica**: son $N+1$ y no N.

Es la regla de *definición* de vértices por atributos (como en el cubo).

Malla poligonal de un cilindro

Paso 3 — Definir los triángulos usando índices

Entre dos anillos, cada par de vértices vecinos forma un cuadrilátero, y cada cuadrilátero se forma mediante **dos triángulos**.

```
for (unsigned i = 0; i < N; ++i)
{
    const unsigned b0 = i,          b1 = i + 1;          // anillo de abajo
    const unsigned t0 = N + 1 + i, t1 = N + 1 + i + 1; // anillo de arriba

    indices.insert(indices.end(), {b0, t1, b1});
    indices.insert(indices.end(), {b0, t0, t1});
}
```

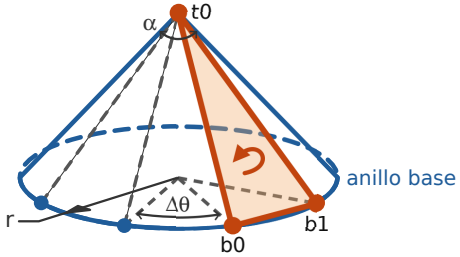
El orden de los tres índices *define* la orientación (normal) de la cara

El orden "natural" —{b0, b1, t1}, en el sentido en que se recorre el bucle— deja la normal de **todos los triángulos al revés**. Y **hoy no pasa nada**: el descarte de caras traseras no está activo.

Malla poligonal de un cono

Datos del cono:

- Radio (ó diámetro)
- Ángulo de conicidad (α)
- Cantidad de gajos (discretización diametral)
- Cantidad de anillos (discretización axial - opcional)



Pasos: (igual que los del cilindro)

1. Definir la ubicación del sistema de referencia del modelo (importante!)
2. Discretizar el **anillo base** para generar los vértices (mismo procedimiento que para cilindro)
3. Usando los *vértices* del anillo base y el vértice del ápice generar un abanico de triángulos

Malla poligonal de un cono

Las normales para el cono

En las caras laterales

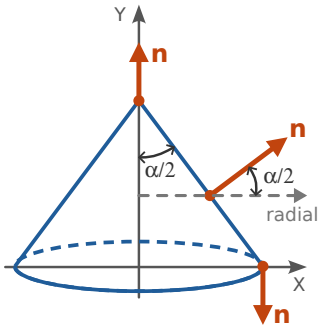
- En el **cilindro** la normal del lateral es radial y sale de normalizar la posición con la componente axial en cero.
- En el **cono** no es radial: está inclinada por el semiángulo de α.

$$\vec{n}_{\text{lat}} = (\cos\theta \cos\tfrac{\alpha}{2},\ \sin\tfrac{\alpha}{2},\ \sin\theta \cos\tfrac{\alpha}{2})$$

En el ápice

- Hay que tomar un criterio porque la iluminación depende de esta definición
- Criterio: No comparte vértices con el lateral: la normal del ápice es **axial** y la del lateral es **la inclinada** de arriba. La regla otra vez: un atributo difiere, el vértice se duplica.

$$\vec{n}_{\text{apex}} = (0,\ 1,\ 0)$$



Un dato que nadie consume no está validado

Hoy la normal se **calcula** y se **guarda**, y no la usa nadie: la iluminación entra el 23-oct.

Una normal mal calculada no da ningún síntoma ahora.

Método de verificación gráfico
Una forma de verificar es "pintar" los vértices usando como color el valor calculado de la normal.

```
// fragment shader, modo de depuracion: dos lineas
// FragColor = vec4(uColor, 1.0);
   FragColor = vec4(vNormal * 0.5 + 0.5, 1.0);   // la normal, como color
```

Cada dirección proporciona un color distinto. Una cara de color plano y uniforme es una normal constante; un degradé suave es una normal que gira.

Actividad de aula

El primer avión para el proyecto se arma con primitivas — las que ya están planteadas y algunas que se agreguen si es necesario —

1. **Fijar los ejes.** Sobre las tres vistas: dónde va el **origen** de la escena y hacia dónde apunta cada eje.
2. **Despiece.** Para cada pieza —fuselaje, nariz, alas, empenaje—: **qué primitiva**, con **qué medidas**, y **qué transformaciones y en qué orden**.
3. **Contar.** ¿Cuántas **mallas distintas** hay que generar? ¿Y cuántas **matrices de modelo** por cuadro? ¿Por qué no coinciden?

En la hoja

Se trabaja sobre la hoja que se reparte, que trae las tres vistas del avión. Las medidas van **relativas al largo del fuselaje** ($L = 1$): no hace falta ningún dato real.

Resumen de GLSL hasta hoy

```
#version 460 core                                   // version de la API requerida
// ----------------- vertex shader -----------------
layout (location = 0) in  vec3 aPos;                // atributo de entrada 1
layout (location = 1) in  vec3 aNormal;             // atributo de entrada 2

out vec3 vNormal;                                   // dato de una etapa a la siguiente

uniform mat4 uModel;                                // HOY: la matriz de este modelo
uniform mat4 uAjuste;                               // HOY: sigue siendo caja negra

void main()
{
    vNormal = aNormal;                              // se copia a la siguiente etapa
    gl_Position = uAjuste * uModel * vec4(aPos, 1.0);
}
// ---------------- fragment shader ----------------
uniform vec3 uColor;                                // HOY: un color por objeto
```

Práctico 03 — Ampliar los módulos existentes

1. **Migrar** `Vertex` a `{posición, normal, coordenadas de textura}`
2. **Agregar** `cylinder()` y `cone()` al módulo `primitives`
3. **Ampliar** `Shader` con los `set_uniform()` que haga falta.
4. **Dibujar dos o tres primitivas** en una escena, cada una con su matriz de modelo y su color.

Si el fuselaje se modela con un cilindro y para achatarlo se lo escala en **la dirección** y **por 0,5**.

¿Qué le pasa a las normales?

¿Siguen siendo perpendiculares a la superficie?